

Stroke Prediction — Model Comparison

Dataset: `healthcare-dataset-stroke-data.csv` (5,110 patient records, binary stroke outcome, ~5% positive).

1. Imports

```
In [1]: import os
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold, cr
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (RandomForestClassifier, GradientBoostingCla
                              VotingClassifier, AdaBoostClassifier)

from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_scor
                              roc_auc_score, roc_curve, confusion_matrix,

from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
from xgboost import XGBClassifier

import sklearn, imblearn, xgboost

sns.set_style('whitegrid')
RNG = 42

print('Imports & setup successful')
print(f' numpy          : {np.__version__}')
print(f' pandas          : {pd.__version__}')
print(f' scikit-learn     : {sklearn.__version__}')
print(f' imbalanced-learn : {imblearn.__version__}')
print(f' xgboost          : {xgboost.__version__}')
print(f' Random seed     : {RNG}')
```

```
Imports & setup successful
numpy          : 2.4.4
pandas         : 3.0.2
scikit-learn   : 1.8.0
imbalanced-learn : 0.14.1
xgboost        : 3.2.0
Random seed    : 42
```

2. Load Data

Tries the local CSV first. If it isn't found, downloads it from Kaggle via `kagglehub`.

```
In [2]: CSV_NAME = 'healthcare-dataset-stroke-data.csv'

if os.path.exists(CSV_NAME):
    df = pd.read_csv(CSV_NAME)
    print(f'Loaded local file: {CSV_NAME}')
else:
    print(f'Local file {CSV_NAME} not found - fetching from Kaggle...')
    import kagglehub
    path = kagglehub.dataset_download('fedesoriano/stroke-prediction-data')
    print('Path to dataset files:', path)
    df = pd.read_csv(os.path.join(path, CSV_NAME))
    print('Loaded from kagglehub')

print('Shape:', df.shape)
df.head()
```

Loaded local file: healthcare-dataset-stroke-data.csv

Shape: (5110, 12)

```
Out[2]:
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Re
0	9046	Male	67.0	0	1	Yes	Private	
1	51676	Female	61.0	0	0	Yes	Self-employed	
2	31112	Male	80.0	0	1	Yes	Private	
3	60182	Female	49.0	0	0	Yes	Private	
4	1665	Female	79.0	1	0	Yes	Self-employed	

3. Quick EDA

```
In [3]: print('Class balance:')
print(df['stroke'].value_counts())
print(f"\nStroke rate: {df['stroke'].mean()*100:.2f}%")

# bmi has 'N/A' strings - convert to numeric
df['bmi'] = pd.to_numeric(df['bmi'], errors='coerce')
print('\nMissing values:')
print(df.isnull().sum()[df.isnull().sum() > 0])
```

```

Class balance:
stroke
0      4861
1       249
Name: count, dtype: int64

```

Stroke rate: 4.87%

```

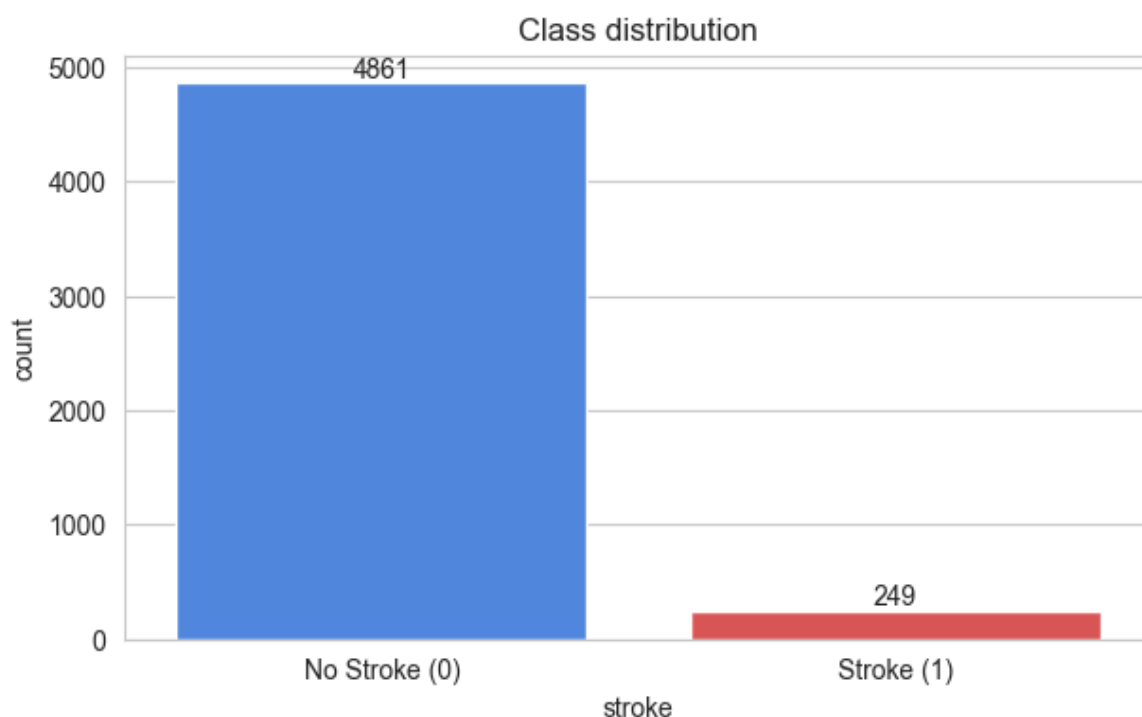
Missing values:
bmi      201
dtype: int64

```

```

In [4]: plt.figure(figsize=(7,4))
ax = sns.countplot(x='stroke', data=df, palette=['#3b82f6', '#ef4444'])
ax.set_xticklabels(['No Stroke (0)', 'Stroke (1)'])
for p in ax.patches:
    ax.annotate(int(p.get_height()), (p.get_x()+p.get_width()/2, p.get_he
plt.title('Class distribution')
plt.show()

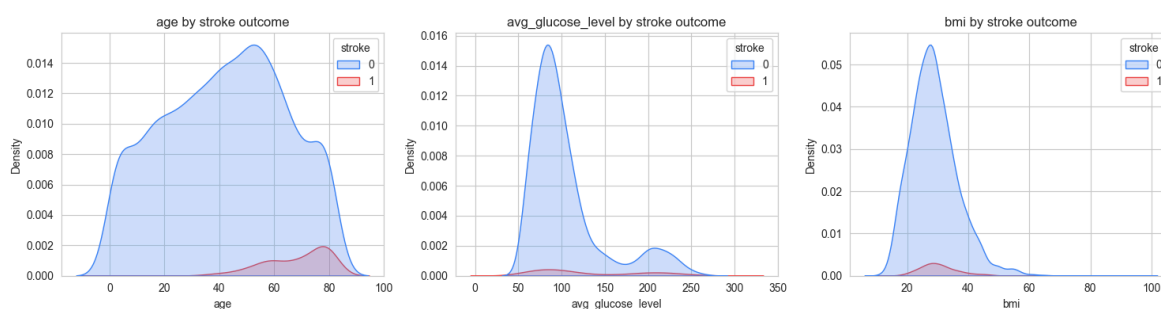
```



```

In [5]: # Numeric feature distributions split by outcome
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, col in zip(axes, ['age', 'avg_glucose_level', 'bmi']):
    sns.kdeplot(data=df, x=col, hue='stroke', fill=True, ax=ax, palette=[
    ax.set_title(f'{col} by stroke outcome')
plt.tight_layout()
plt.show()

```



4. Cleaning

```
In [6]: # Drop the single 'Other' gender row and the non-predictive id column
df = df[df['gender'] != 'Other'].copy()
df = df.drop(columns=['id'])
print('Cleaned shape:', df.shape)
```

Cleaned shape: (5109, 11)

5. Preprocessing Pipeline

Numeric → median impute + scale. Categorical → mode impute + one-hot. Binary → pass-through.

```
In [7]: y = df['stroke']
X = df.drop(columns=['stroke'])

num_cols = ['age', 'avg_glucose_level', 'bmi']
cat_cols = ['gender', 'ever_married', 'work_type', 'Residence_type', 'smoking']
bin_cols = ['hypertension', 'heart_disease']

numeric_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
categorical_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])
preprocessor = ColumnTransformer([
    ('num', numeric_pipe, num_cols),
    ('cat', categorical_pipe, cat_cols),
    ('bin', 'passthrough', bin_cols)
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RNG, stratify=y
)
print(f'Train: {X_train.shape} Test: {X_test.shape}')
```

Train: (4087, 10) Test: (1022, 10)

6. Train 9 Models

Each model is wrapped in a pipeline with SMOTE applied only to the training fold.

```
In [8]: voting_estimators = [
    ('lr', LogisticRegression(max_iter=1000, random_state=RNG)),
    ('rf', RandomForestClassifier(n_estimators=300, random_state=RNG, n_j
    ('gb', GradientBoostingClassifier(random_state=RNG)),
]

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_stat
    'Decision Tree': DecisionTreeClassifier(max_depth=6, random_st
```

```

'Random Forest': RandomForestClassifier(n_estimators=300, rand
'AdaBoost': AdaBoostClassifier(n_estimators=300, random_s
'Gradient Boosting': GradientBoostingClassifier(random_state=RNG),
'XGBoost': XGBClassifier(n_estimators=300, learning_rate
                    random_state=RNG, eval_metric='
'Voting Classifier': VotingClassifier(estimators=voting_estimators
'SVM (RBF kernel)': SVC(kernel='rbf', probability=True, random_st
'Neural Network (MLP)': MLPClassifier(hidden_layer_sizes=(64,32), act
                    max_iter=300, random_state=RNG,
}

results, trained, roc_data = [], {}, {}
for name, clf in models.items():
    pipe = ImbPipeline([
        ('prep', preprocessor),
        ('smote', SMOTE(random_state=RNG)),
        ('clf', clf)
    ])
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    y_proba = pipe.predict_proba(X_test)[:,1]
    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred, zero_division=0),
        'Recall': recall_score(y_test, y_pred),
        'F1-Score': f1_score(y_test, y_pred),
        'ROC-AUC': roc_auc_score(y_test, y_proba),
    })
    trained[name] = pipe
    roc_data[name] = roc_curve(y_test, y_proba)
    print(f' {name:24s} AUC={results[-1]["ROC-AUC"]:.3f} Rec={results[

```

Logistic Regression	AUC=0.840	Rec=0.800
Decision Tree	AUC=0.747	Rec=0.560
Random Forest	AUC=0.770	Rec=0.120
AdaBoost	AUC=0.813	Rec=0.560
Gradient Boosting	AUC=0.803	Rec=0.440
XGBoost	AUC=0.776	Rec=0.200
Voting Classifier	AUC=0.824	Rec=0.460
SVM (RBF kernel)	AUC=0.765	Rec=0.440
Neural Network (MLP)	AUC=0.722	Rec=0.220

7. Leaderboard

```

In [9]: leaderboard = pd.DataFrame(results).sort_values('ROC-AUC', ascending=False)
leaderboard.style.format({c: '{:.3f}' for c in ['Accuracy', 'Precision', 'Re

```

Out [9]:

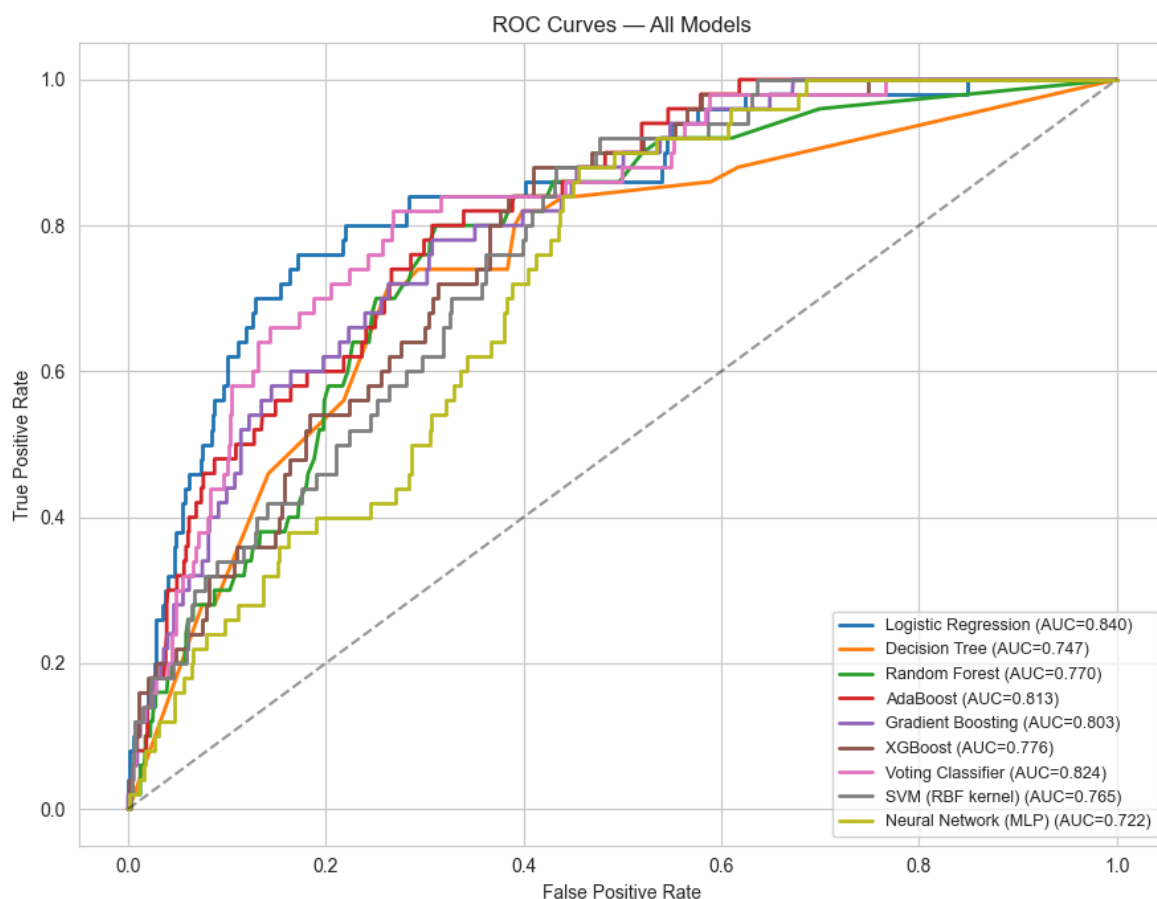
	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Logistic Regression	0.736	0.133	0.800	0.229	0.840
1	Voting Classifier	0.879	0.192	0.460	0.271	0.824
2	AdaBoost	0.835	0.160	0.560	0.249	0.813
3	Gradient Boosting	0.877	0.183	0.440	0.259	0.803
4	XGBoost	0.932	0.250	0.200	0.222	0.776
5	Random Forest	0.935	0.214	0.120	0.154	0.770
6	SVM (RBF kernel)	0.792	0.106	0.440	0.171	0.765
7	Decision Tree	0.771	0.117	0.560	0.193	0.747
8	Neural Network (MLP)	0.893	0.136	0.220	0.168	0.722

8. ROC Curves

```

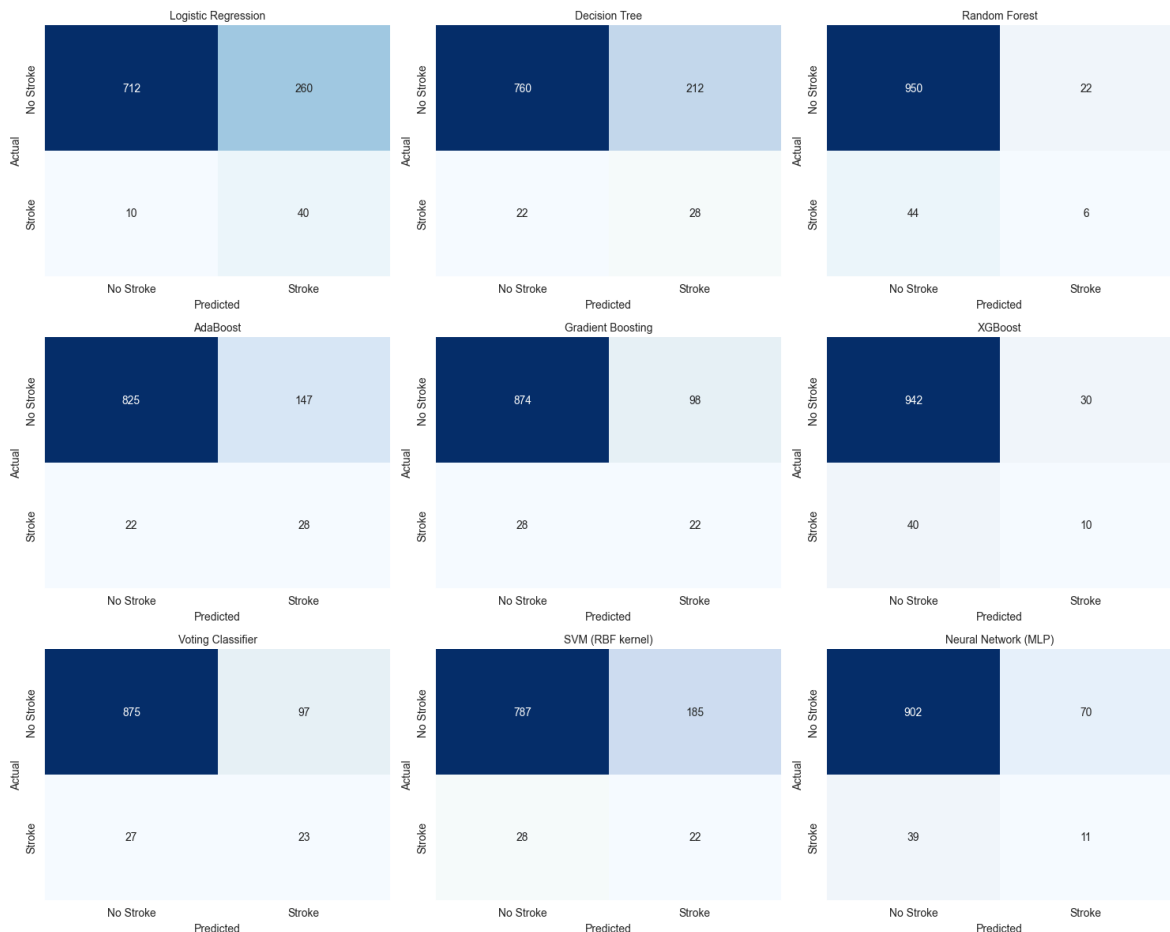
In [10]: plt.figure(figsize=(9,7))
for name, (fpr, tpr, _) in roc_data.items():
    auc = next(r['ROC-AUC'] for r in results if r['Model']==name)
    plt.plot(fpr, tpr, label=f'{name} (AUC={auc:.3f})', linewidth=2)
plt.plot([0,1],[0,1], 'k--', alpha=0.4)
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('ROC Curves - All Models')
plt.legend(loc='lower right', fontsize=9); plt.tight_layout(); plt.show()

```



9. Confusion Matrices

```
In [11]: fig, axes = plt.subplots(3, 3, figsize=(15,12))
axes = axes.flatten()
for i, (name, pipe) in enumerate(trained.items()):
    cm = confusion_matrix(y_test, pipe.predict(X_test))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[i], cbar=False,
                xticklabels=['No Stroke', 'Stroke'], yticklabels=['No Stro
    axes[i].set_title(name, fontsize=10)
    axes[i].set_xlabel('Predicted'); axes[i].set_ylabel('Actual')
for j in range(len(trained), len(axes)): axes[j].axis('off')
plt.tight_layout(); plt.show()
```



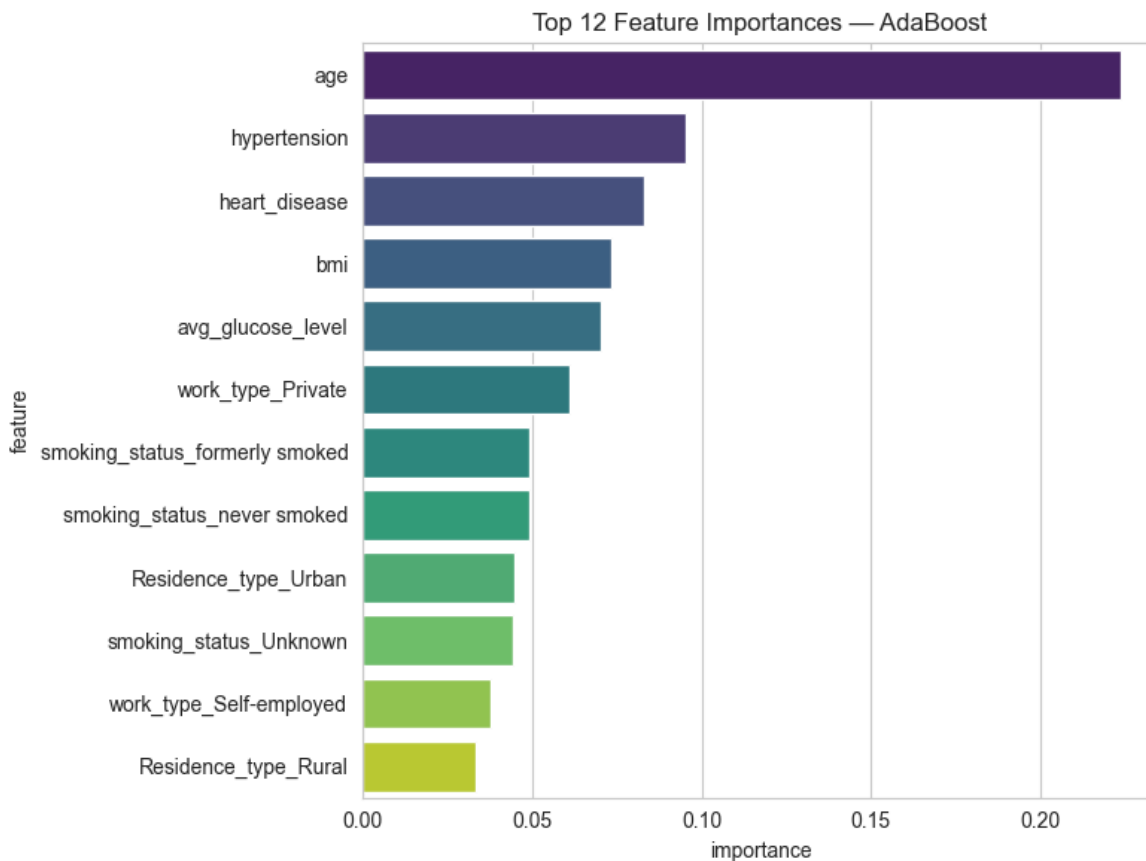
10. Feature Importance

Uses the highest-ranked tree-based model from the leaderboard.

```
In [12]: tree_models = ['XGBoost', 'Random Forest', 'Gradient Boosting', 'AdaBoost', '
fi_name = next((m for m in leaderboard['Model'] if m in tree_models), 'Ra
fi_pipe = trained[fi_name]
fi_prep = fi_pipe.named_steps['prep']; fi_clf = fi_pipe.named_steps['clf']
ohe = fi_prep.named_transformers_['cat'].named_steps['onehot']
feat_names = num_cols + list(ohe.get_feature_names_out(cat_cols)) + bin_c
fi_df = pd.DataFrame({'feature': feat_names, 'importance': fi_clf.feature
                     .sort_values('importance', ascending=False).head(12)})

plt.figure(figsize=(8,6))
sns.barplot(data=fi_df, x='importance', y='feature', palette='viridis')
plt.title(f'Top 12 Feature Importances - {fi_name}')
```

```
plt.tight_layout(); plt.show()
fi_df
```



Out [12]:

	feature	importance
0	age	0.223623
18	hypertension	0.095464
19	heart_disease	0.083082
2	bmi	0.073424
1	avg_glucose_level	0.070255
9	work_type_Private	0.060872
15	smoking_status_formerly smoked	0.049273
16	smoking_status_never smoked	0.049185
13	Residence_type_Urban	0.044823
14	smoking_status_Unknown	0.044180
10	work_type_Self-employed	0.037966
12	Residence_type_Rural	0.033346

11. Classification Report — Best Model

```
In [13]: best_name = leaderboard.iloc[0]['Model']
best_pipe = trained[best_name]
print(f'BEST MODEL: {best_name}\n')
print(classification_report(y_test, best_pipe.predict(X_test), target_name='stroke_prediction'))
```


BEST MODEL: Logistic Regression

	precision	recall	f1-score	support
No Stroke	0.99	0.73	0.84	972
Stroke	0.13	0.80	0.23	50
accuracy			0.74	1022
macro avg	0.56	0.77	0.53	1022
weighted avg	0.94	0.74	0.81	1022

Deeper Analysis on the Best Model

12. Threshold Tuning

The default 0.5 threshold is rarely optimal for imbalanced healthcare data. Sweep the threshold to see how precision, recall and F1 trade off.

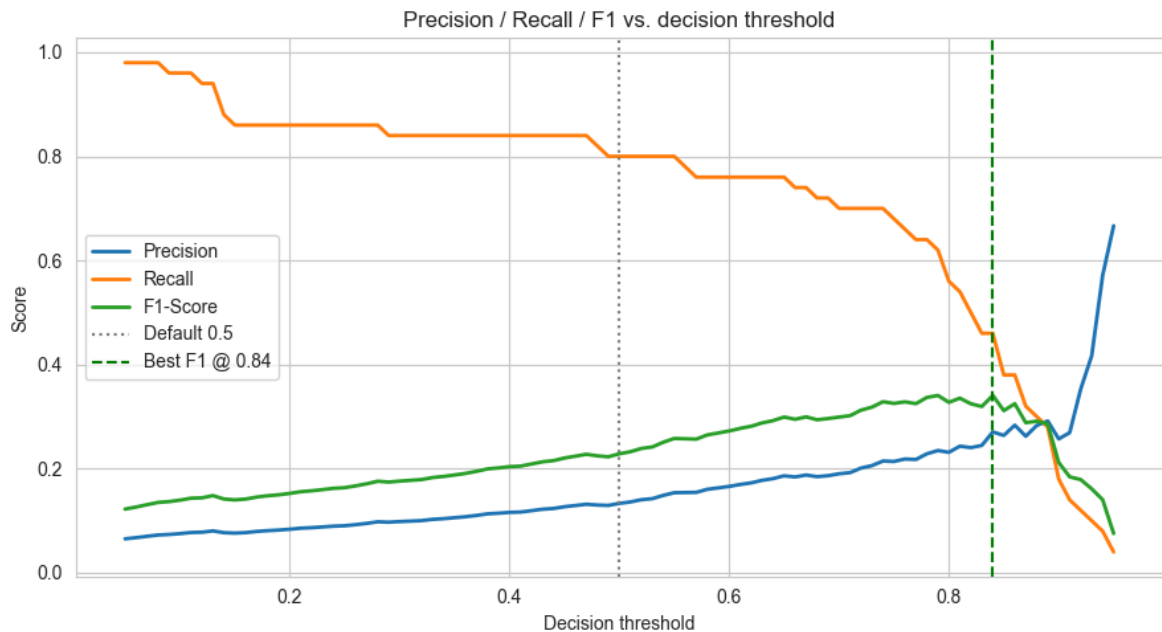
```
In [14]: y_proba_best = best_pipe.predict_proba(X_test)[: , 1]

thresholds = np.linspace(0.05, 0.95, 91)
prec_list, rec_list, f1_list = [], [], []
for t in thresholds:
    pred = (y_proba_best >= t).astype(int)
    prec_list.append(precision_score(y_test, pred, zero_division=0))
    rec_list.append(recall_score(y_test, pred))
    f1_list.append(f1_score(y_test, pred))

best_f1_idx = int(np.argmax(f1_list))
best_f1_th = thresholds[best_f1_idx]
hi_recall_idx = max(next(i for i, r in enumerate(rec_list) if r < 0.85) -
print(f'Best-F1 threshold      : {best_f1_th:.2f} (P={prec_list[best_f1_idx]:.2f}, R={rec_list[best_f1_idx]:.2f}, F1={f1_list[best_f1_idx]:.2f})')
print(f'Recall>=0.85 threshold : {thresholds[hi_recall_idx]:.2f} (P={prec_list[hi_recall_idx]:.2f}, R={rec_list[hi_recall_idx]:.2f}, F1={f1_list[hi_recall_idx]:.2f})')

Best-F1 threshold      : 0.84 (P=0.271, R=0.460, F1=0.341)
Recall>=0.85 threshold : 0.28 (P=0.098, R=0.860)
```

```
In [15]: plt.figure(figsize=(9,5))
plt.plot(thresholds, prec_list, label='Precision', linewidth=2)
plt.plot(thresholds, rec_list, label='Recall', linewidth=2)
plt.plot(thresholds, f1_list, label='F1-Score', linewidth=2)
plt.axvline(0.5, color='grey', linestyle=':', label='Default 0.5')
plt.axvline(best_f1_th, color='green', linestyle='--', label=f'Best F1 @ {best_f1_th:.2f}')
plt.xlabel('Decision threshold'); plt.ylabel('Score')
plt.title('Precision / Recall / F1 vs. decision threshold')
plt.legend(loc='best'); plt.tight_layout(); plt.show()
```



13. 5-fold Stratified Cross-Validation

Confirms results are stable across different splits — not an artefact of a single train/test split. Reported on the top-3 models from the leaderboard.

```
In [16]: # top 3 models from the leaderboard
top3 = leaderboard['Model'].head(3).tolist()
cv_model_map = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=RNG),
    'Decision Tree': DecisionTreeClassifier(max_depth=6, random_state=RNG),
    'Random Forest': RandomForestClassifier(n_estimators=300, random_state=RNG),
    'AdaBoost': AdaBoostClassifier(n_estimators=300, random_state=RNG),
    'Gradient Boosting': GradientBoostingClassifier(random_state=RNG),
    'XGBoost': XGBClassifier(n_estimators=300, learning_rate=0.1, random_state=RNG, eval_metric='logloss'),
    'Voting Classifier': VotingClassifier(estimators=voting_estimators),
    'SVM (RBF kernel)': SVC(kernel='rbf', probability=True, random_state=RNG),
    'Neural Network (MLP)': MLPClassifier(hidden_layer_sizes=(64,32), activation_function='tanh', max_iter=300, random_state=RNG),
}

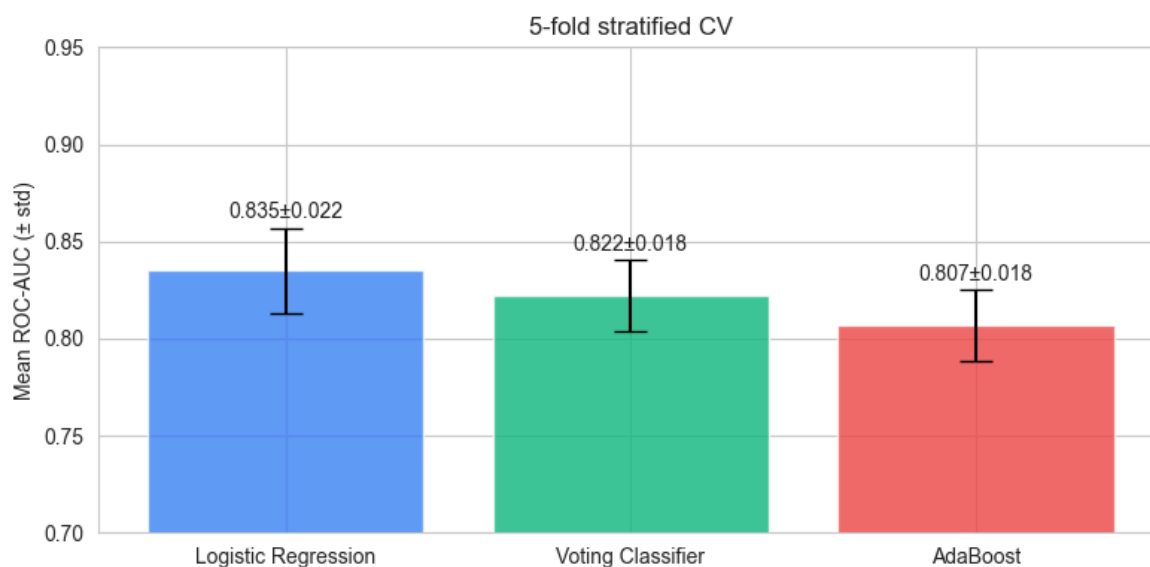
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RNG)
cv_results = []
for name in top3:
    clf = cv_model_map[name]
    pipe = ImbPipeline([
        ('prep', preprocessor),
        ('smote', SMOTE(random_state=RNG)),
        ('clf', clf)
    ])
    scores = cross_val_score(pipe, X, y, scoring='roc_auc', cv=cv, n_jobs=-1)
    cv_results.append({'Model': name,
                      'Mean ROC-AUC': scores.mean(),
                      'Std ROC-AUC': scores.std(),
                      'Folds': np.round(scores, 4).tolist()})
print(f' {name:24s} AUC = {scores.mean():.3f} ± {scores.std():.3f}')
pd.DataFrame(cv_results)
```

Logistic Regression AUC = 0.835 ± 0.022
 Voting Classifier AUC = 0.822 ± 0.018
 AdaBoost AUC = 0.807 ± 0.018

Out [16]:

	Model	Mean ROC-AUC	Std ROC-AUC	Folds
0	Logistic Regression	0.834961	0.022099	[0.8194, 0.824, 0.8564, 0.8658, 0.8091]
1	Voting Classifier	0.822044	0.018487	[0.803, 0.8262, 0.8424, 0.8405, 0.7981]
2	AdaBoost	0.806593	0.018349	[0.7814, 0.827, 0.8125, 0.8234, 0.7888]

```
In [17]: plt.figure(figsize=(8,4))
means = [r['Mean ROC-AUC'] for r in cv_results]
stds = [r['Std ROC-AUC'] for r in cv_results]
xpos = np.arange(len(cv_results))
plt.bar(xpos, means, yerr=stds, capsize=8, color=['#3b82f6', '#10b981', '#e91e63'])
plt.xticks(xpos, [r['Model'] for r in cv_results])
plt.ylabel('Mean ROC-AUC (± std)'); plt.title('5-fold stratified CV')
plt.ylim(0.7, 0.95)
for i,(m,s) in enumerate(zip(means, stds)):
    plt.text(i, m+s+0.005, f'{m:.3f}±{s:.3f}', ha='center', fontsize=10)
plt.tight_layout(); plt.show()
```



14. Per-Subgroup Fairness Audit

Does the model perform equally well across age bands, genders, and work types?
 Flags any subgroup where performance drops below the overall AUC.

```
In [18]: test = X_test.copy()
test['y_true'] = y_test.values
test['y_proba'] = y_proba_best
test['y_pred'] = (y_proba_best >= 0.5).astype(int)
test['age_band'] = pd.cut(test['age'], bins=[0,30,50,70,120], labels=['<30', '30-50', '50-70', '70-120'])

rows = []
for col, label in [('gender', 'Gender'), ('age_band', 'Age band'), ('work_type', 'Work type')]:
```

```

for group, g in test.groupby(col, observed=True):
    if g['y_true'].nunique() < 2:
        auc, rec = np.nan, np.nan
    else:
        auc = roc_auc_score(g['y_true'], g['y_proba'])
        rec = recall_score(g['y_true'], g['y_pred'])
    rows.append({'Attribute': label, 'Group': str(group),
                'N': len(g),
                'Stroke rate %': round(g['y_true'].mean()*100, 2),
                'ROC-AUC': round(auc,3) if not np.isnan(auc) else No,
                'Recall @0.5': round(rec,3) if not np.isnan(rec) els

fairness_df = pd.DataFrame(rows)
fairness_df

```

Out[18]:

	Attribute	Group	N	Stroke rate %	ROC-AUC	Recall @0.5
0	Gender	Female	597	4.86	0.846	0.828
1	Gender	Male	425	4.94	0.833	0.762
2	Age band	<30	313	0.32	0.471	0.000
3	Age band	30-49	281	2.49	0.484	0.000
4	Age band	50-69	292	3.77	0.755	0.818
5	Age band	70+	136	22.79	0.598	1.000
6	Work type	Govt_job	129	3.88	0.915	1.000
7	Work type	Never_worked	6	0.00	NaN	NaN
8	Work type	Private	600	5.67	0.845	0.765
9	Work type	Self-employed	155	6.45	0.710	0.900
10	Work type	children	132	0.76	0.840	0.000

In [19]:

```

overall_auc = leaderboard.iloc[0]['ROC-AUC']
plot_df = fairness_df.dropna(subset=['ROC-AUC']).copy()
plot_df['label'] = plot_df['Attribute'] + ': ' + plot_df['Group']
colors = plot_df['Attribute'].map({'Gender': '#3b82f6', 'Age band': '#10b981',

plt.figure(figsize=(10,5))
plt.barh(plot_df['label'], plot_df['ROC-AUC'], color=colors)
plt.axvline(overall_auc, color='grey', linestyle='--', label=f'Overall AU
plt.xlabel('ROC-AUC'); plt.title('Per-subgroup ROC-AUC')
plt.legend(loc='lower right'); plt.tight_layout(); plt.show()

```

